

Twin Primes and Siegel Zeros: a Fulcrum

Jason Hickey

Independent researcher

ORCID [0000-0003-0300-021X](https://orcid.org/0000-0003-0300-021X)

Preprint, 2 October 2026

Abstract. Every theorem in this paper is verified by the Lean 4 kernel over mathlib, with axiom base exactly `{propext, Classical.choice, Quot.sound}`; nothing rests on informal argument. We introduce the *fulcrum*: a single hypothesis—one fixed quality constant C^* , one zero per unbounded window—under which, together with an explicitly stated engine hypothesis, Siegel zeros produce infinitely many twin primes; the zero’s reality is derived rather than assumed, and the fulcrum is proved minimal for the engine that consumes it. Around it we prove kernel-checked zero-free regions for ζ , a Deuring–Heilbronn repulsion contract, an exchange-rate neutrality theorem pricing transfer-based routes to twins, and the parity gap as a mathematical object. The corpus is 324,724 lines of Lean across 751 files, produced by the *Salt method*, which is described as a contribution in its own right.

Keywords: twin primes, Siegel zeros, zero-free regions, parity problem, Chowla conjecture, formalization of mathematics, Lean theorem prover.

MSC 2020: Primary 11M26; Secondary 68V20, 11N35.

Overview

Heath-Brown proved [4] that infinitely many Siegel zeros would force infinitely many twin primes; the zero-free-region landscape such arguments inhabit descends from Littlewood [5]. Machine-checked mathematics at research scale has its landmark in the Kepler verification [3], over substrates such as mathlib [6]. This overview places the paper in that context and states its results; the abstract above is a summary.

Every theorem in this paper is verified by the Lean 4 kernel over mathlib, with axiom base exactly `{propext, Classical.choice, Quot.sound}`; nothing rests on informal argument. The mathematics—the designs, the proofs, and their formalization—is joint work of the author and Claude (Anthropic’s Fable 5 and Opus models), carried out under the author’s direction and ratification; the commit ledger records the collaboration line by line.

We introduce the *fulcrum*: a single hypothesis—one fixed quality constant C^* , one zero per unbounded window—under which, together with an explicitly stated engine hypothesis, Siegel zeros produce infinitely many twin primes; the fulcrum is proved minimal for the engine that consumes it, with the zero’s reality *derived* rather than assumed. The fulcrum was not transcribed from the literature: it was extracted from a formal corpus by demand-audited hypothesis minimization, a mode of discovery we argue is impossible over prose proofs. Around it we prove, kernel-checked: a Heath-Brown–type dichotomy (twin primes, or no Siegel zeros) with its engine-conditional horn stated explicitly; the first formalization, in any public artifact as of 2026-08-17,¹ of any zero-free region for ζ of Littlewood strength (1922) and the first such power-saving region ($\theta = 3/4$) in any proof assistant; a Deuring–Heilbronn repulsion

¹Priority statements in this paper are *surveyed as-of* claims, not assertions of absence. Surveys: `docs/priority-survey-2026-08-11.md` and the pre-submission re-run `docs/priority-survey-2026-08-17-delta.md`, both in this repository (method: adversary-cited, five independent lanes each; two of the re-run’s changed verdicts rest on our own local kernel checks of competitors’ public bytes, not on their claims). This repository is public as of 2026-08-16; the earliest third-party-verifiable date for its contents is 2026-08-14, the submission timestamp of the companion methods paper; internal commit dates before that are reported as internal records, never as public priority. Our own landing dates are given in §3 and the results register; the survey re-runs once more at submission.

contract; the exact-identity keystone of the Heath-Brown engine, proven unconditional; the *exchange-rate wall*, a neutrality theorem over arbitrary real sign functions that prices every transfer-based route to twins; and the *parity gap* as a mathematical object: a predicate Z with a kernel-checked theorem that no twin-sufficient, parity-invariant completion exists, together with a census placing ten headline theorems of the corpus inside the parity-invariant cone. As a byproduct, the logarithmic two-point Chowla statement—a theorem of Tao—is machine-checked conditionally on a single named uniformity hypothesis. The corpus is 324,724 lines of Lean across 751 files; the method that produced it—the *Salt method*—kept a public ledger of every design error caught (256 numbered entries) against zero wrong proofs, and is described as a contribution in its own right.

1 Introduction

The twin prime conjecture is not merely unproven; the routes toward it are walled by a structural obstruction—the parity barrier—that has never itself been a theorem, and guarded by a ghost—the Siegel zero—that has never been an object one could bargain with precisely. Progress of the conditional kind (Heath-Brown’s 1983 theorem that infinitely many Siegel zeros force infinitely many twin primes [4]) trades in hypotheses whose exact strength is scattered through pages of estimates: what, *precisely*, must one believe about exceptional zeros to believe in twin primes? Prose proofs cannot answer such questions exhaustively, because a prose proof hides its demand.

A formal proof *is* its demand. This paper’s central object, the **fulcrum**, was found by exploiting exactly that: we formalized a Heath-Brown-type engine, enumerated—mechanically, exhaustively—every site at which the Siegel-zero hypothesis is consumed and at what strength, and then weakened the hypothesis axis by axis to the audited demand, with the Lean kernel refereeing each weakening instantly. The fixed point of that process is the weakest twin-producing Siegel hypothesis in the surveyed record (Definition 4.1); the process itself, *demand-audited hypothesis minimization*, is—we argue—a new mode of mathematical discovery, and it is the sense in which this paper’s formalization is an instrument rather than a notary.

The results divide into four movements. First, the *banners* (§3): unconditional analytic theorems, two of them formalized for the first time in any public artifact as of 2026-08-17: a Littlewood-strength zero-free region for ζ and the first such power-saving region in any proof assistant. (The Vinogradov mean value theorem that feeds the latter carries its own dated qualification, stated at Remark 3.6.) Second, the *fulcrum and the dichotomy* (§§4–6): the minimized hypothesis, the kernel-checked dichotomy it induces, and the exact-identity keystone of the engine, now proven unconditional. Third, the *negative space* (§§7–8): the exchange-rate wall and the parity gap, which together map—as theorems, not folklore—why the obvious routes die. Fourth, the *method* (§10): the Salt method, an adversarial division of mathematical labor among human direction, frontier-model design, fleet-model execution and refutation, and kernel verification, whose public error *ledger* — an append-only record in the repository of every design error caught, numbered and repaired, whose entries we cite as (ledger # N) — stands at 256 numbered catches against zero wrong proofs and is offered as evidence that the standard of the first sentence of the abstract is achievable at research scale and research speed. All of it was produced by the Salt method (§10).

Throughout, conditionality is a first-class citizen: every conditional statement names its exact residual hypothesis, and Table 1 lists all of them in one place. The paper makes no unconditional claim about twin primes.

2 The corpus and the referee

The corpus is 324,724 lines of Lean 4 over mathlib [6], in 751 files, built by 1034 commits; a full build kernel-checks roughly 9,400 compilation jobs. The skeptic’s question —*how do we really know?*—splits

into what is mechanically excluded and what is irreducibly a human question, and we answer it in that order.

2.1 Mechanically excluded

A `sorry` anywhere in a dependency tree—ours or `mathlib`’s—introduces the axiom `sorryAx`, which propagates transitively and surfaces in the axiom closure of every downstream theorem. Our `#audit_axioms` blocks print that closure for every headline statement: it is exactly `{propext, Classical.choice, Quot.sound}`, the three classical axioms of `mathlib` practice. A hidden gap cannot hide. Tactic bugs cannot certify falsehoods: tactics emit proof terms that the kernel re-checks independently, so the trust surface is the kernel alone, a deliberately small core; we additionally forbid `native_decide` (which would trust the compiler, visibly, as a fourth axiom). The kernel itself is the one residual mechanical trust, and it is checkable: before submission every declaration is replayed through `leanchecker`, the toolchain’s standalone proof checker, independently of the elaborator and build cache that produced it. The replay does not remove the kernel from the trusted base—checker and kernel share an implementation—but it excludes elaborator and cache faults, which is where the practical risk lives. One consequence for the reader: every numbered statement in this paper—theorem, proposition, corollary—is kernel-checked; the labels grade importance, never epistemic status. Verification at this scale is not itself new—the formal proof of the Kepler conjecture [3] is the standing landmark—but the object here is a research program in progress rather than a settled theorem, which is what makes the demand audit of §4 possible at all.

2.2 The specification gap

What no formal system can check is whether our *statements* mean what our prose claims. This gap is irreducible—it is the map–territory boundary—so we engineer against it. Definitions bottom out in `mathlib`’s communal objects (`Nat.Prime`, `DirichletCharacter`, `LFunction`), vetted by thousands of independent uses. Vacuity is an attacked property, not an assumed one: adversarial review passes carry mandatory degenerate-instance checks, and where a definition provably trivializes outside its intended window, the degeneracy is *stated in Lean* beside it (see the grade guard of §8). Non-vacuity is proved by witness: the fulcrum’s zero-reality is derived from our own zero-free region (Theorem 4.2); the parity census instantiates its cone with ten corpus theorems; classical corollaries are re-derived from our statements, which a misstated premise would break. Finally, Appendix A renders every headline statement in Lean and in prose side by side, reducing the entire correctness question of the corpus to a finite, public, refereeable translation audit. “Absolutely” is not on offer anywhere in mathematics; what is on offer here is a doubt surface consisting of one small kernel, three axioms, and a one-page table.

3 The four banners

Four unconditional theorems anchor the corpus. We do not assert that they were unformalized—an absence no repository search can establish, as the survey behind this paper’s priority statements found to its cost, twice—and cite the adversaries instead: the canonical public Lean formalization of Bombieri–Vinogradov proves its main theorem modulo a Siegel–Walfisz statement *axiomatized* in its own source, and says so; its companion large-sieve input, recorded as an axiom by the 2026-08-11 survey, is discharged by a proof (primitive-character scope, Gallagher’s route) public since 2026-07-09 as that repository’s first pull request, merged 2026-08-04—which the 2026-08-17 re-run kernel-checked clean from its public bytes, and which therefore precedes this repository’s public dates. The two proofs live on sibling branches, not yet merged. As of 2026-08-17 no public artifact other than the present corpus proves unconditional Bombieri–Vinogradov; Siegel–Walfisz is the wall—a declared axiom on all four

branches of the canonical repository and in a second project, a sorry or a self-described scaffold in the others the survey read.

Theorem 3.1 (Littlewood-strength zero-free region, 1922). *There exist effective constants such that $\zeta(s) \neq 0$ in a region of Littlewood type $\sigma \geq 1 - c \frac{\log \log |t|}{\log |t|}$ for $|t|$ large.*

`Salt.Vk.zeta_zero_free_region_littlewood`

Theorem 3.2 (A power-saving region, $\theta = 3/4$). *There exist effective constants such that $\zeta(s) \neq 0$ for $\sigma \geq 1 - c (\log |t|)^{-3/4} (\log \log |t|)^{-O(1)}$, $|t|$ large: a Vinogradov–Korobov-type region at exponent $\theta = 3/4$.*

`Salt.Vk.zeta_zero_free_region_pow`

As of the 2026-08-17 survey re-run, Theorem 3.2 is the first zero-free region with a power saving in the exponent verified in any public proof-assistant artifact, and Theorem 3.1 the first formalization of the 1922 Littlewood region [5]; the strongest previously formalized regions were of classical de la Vallée Poussin shape. Our exponent convention for zero-free regions, stated so that the claim is checkable: θ is the power of $\log |t|$ in the denominator of the zero-free width, so the de la Vallée Poussin region $\sigma \geq 1 - c/\log |t|$ sits at $\theta = 1$, the Vinogradov–Korobov region of the prose literature $\sigma \geq 1 - c (\log |t|)^{-2/3} (\log \log |t|)^{-1/3}$ at $\theta = 2/3$, and *power saving* means $\theta < 1$. Theorem 3.2 lands at $\theta = 3/4$: strictly between de la Vallée Poussin and the classical $2/3$, which on the same survey no public proof-assistant artifact attains; we do not claim the $2/3$ exponent. The proofs formalize the Vinogradov mean value theorem (`Salt.Vmvt.vmvt`) and route it through an exponential-sum-to-growth bridge (`Salt.Vk.vk_block_core`), with an explicit strip family at absolute constant $C = 4096$.

Theorem 3.3 (Deuring–Heilbronn repulsion, contract form). *There are explicit constants $b = 680$, $k = 14$ and $c > 0$ such that: for every primitive quadratic character $\chi \bmod q$, $q \geq 2$, every real zero $\beta_0 \in (\frac{1}{2}, 1)$ of $L(s, \chi)$, and every non-real zero ρ of $L(s, \chi)$ with $\frac{16}{17} \leq \operatorname{Re} \rho \leq \beta_0$ — at every height —*

$$1 - \beta_0 \geq \frac{c (q(|\operatorname{Im} \rho| + 2))^{-b(1 - \operatorname{Re} \rho)}}{(\log(q(|\operatorname{Im} \rho| + 2)) + 2)^k}.$$

The closer the real zero sits to 1, the farther every companion zero is pushed from the 1-line: the Deuring–Heilbronn repulsion phenomenon, in ordered two-zero contract form with every constant explicit.

`Salt.SW.dh_repulsion_tall`

Remark 3.4 (The constant, and what sharpening it buys). The height restriction $|\operatorname{Im} \rho| \leq 1$ is not needed: it was removed from the contract outright rather than widened to a parameter, and the height-bounded twin survives beside it at the same b and k (`Salt.SW.dh_repulsion_ordered`). The constant c is not a single symbol but an explicit minimum of ten arms, each the price of one row of the underlying estimate; in the unrestricted form every arm is free of the auxiliary ζ -growth constant, so the tower prices exactly:

$$\log(1/c) = 86.2267 \quad \text{at } c_0 = 1/126848,$$

c_0 being the corpus’s own zero-free-region constant, and the binding arm being $(c_0/32)^{17/3}$ — that is, the repulsion constant is now limited by our zero-free region rather than by the estimate chain that consumes it. The figure was $\log(1/c) = 631.5764$ when the contract first landed; two sharpening waves (replacing a uniform exponent by each row’s own Hölder floor, and retiring a 2^{-250} arm) brought it to the value above *with no parameter changed and no statement outside the two contract files touched*. We record what this does not buy: the dichotomy of §5 is stated for every $C > 0$, so no threshold in this paper moves with the size of c ; the gain is explicit-constant hygiene, and it matters only where a downstream hypothesis packet must be shown non-empty.

Theorem 3.5 (Vinogradov mean value theorem). *The mean value estimate for Weyl sums in the range needed by Theorem 3.2, with explicit constants.*

Salt.Vmvt.vmvt

Remark 3.6 (A dated qualification). For this theorem the as-of dating requires more than the survey’s default as-of form—a dated adversary citation: the `lean-lang.org` evaluation board records its Vinogradov-mean-value problem—stated there in full main-conjecture form—as solved on 2026-08-16 by a third-party submission (an automated system), verified by the board’s own same-statement comparator (kernel-accepted, axioms within the standard three), with the solution source private and its publication announced for 2026-08-31. No public artifact proves the theorem as of 2026-08-17, but a kernel-grade, dated counter-attestation exists and its artifact is expected to be public before review of this paper completes. We therefore attach no priority weight to Theorem 3.5: it appears here as the input that Theorem 3.2 consumes, and the survey delta records the board entry as the adversary citation.

Around these sit an unconditional Siegel–Walfisz theorem, a Bombieri–Vinogradov theorem, a machine-checked Chen’s theorem with the unconditional corollary $\Omega(p(p+2)) \leq 3$ for infinitely many primes p (`Salt.Fulcrum.chen_omega_prod_le_three`), and the unconditional $\Omega(n(n+2)) \leq 20$ infinitely often (`Salt.BrunLower.twin_almost_prime`); these are inherited infrastructure from the same program and are indexed in Appendix A.

4 The fulcrum

4.1 The discovery

The Heath-Brown engine [4] consumes a Siegel-zero hypothesis at many sites: as a lower bound forcing L -values small, as a localization of one zero, as a quality bound entering a sieve level, as a reality assumption. In prose, the totality of this demand is not enumerable; in the formal corpus it is a finite list of typed consumption sites, each with an exact strength. The Fulcrum Hunt (three adversarial passes over the corpus, recorded in the repository’s exploration ledger) audited every site, then weakened the hypothesis along each axis until the audited demand was met exactly—each candidate weakening compiled or refuted by the kernel within minutes. Three collapses survived: the quality constant collapsed from a universally quantified family to a *single fixed* C^* ; the zero’s *reality* moved from hypothesis to conclusion (derived from our own zero-free region); and the *quantity* collapsed to one zero per unbounded window—no asymptotic family, no density, no infinitude of characters beyond what one zero per window forces.

Definition 4.1 (The fulcrum). *For $C > 0$, `FulcrumQualityMin C` asserts: for every natural number Q there exist a modulus $q > Q$, a primitive quadratic character $\chi \bmod q$, and a zero ρ of $L(\cdot, \chi)$ with $\|1 - \rho\| \cdot C \log q \leq 1$. The fulcrum hypothesis is*

$$F := \text{FulcrumQualityMin } C^*$$

for the single explicit constant $C^ = \max(C^{(1)}, 2/c_0)$, where $c_0 = 1/126848$ is the zero-free-region constant of the corpus, kernel-certified at that numeral (`zero_free_region_all_numeral`); the $2/c_0$ arm of C^* is exactly 253696, and the fulcrum’s reality theorem is instantiated at that bare threshold (`fulcrum_zero_real_numeral`).*

Salt.Fulcrum.FulcrumQualityMin

Theorem 4.2 (Reality is derived). *Any zero witnessing the fulcrum at quality C^* is real, and lies in the exceptional position of Siegel-zero theory.*

Salt.Fulcrum.fulcrum_zero_real

Remark 4.3. The threshold Q is a bare cursor: the quantifier $\forall Q \exists q > Q$ asserts exactly that the qualifying moduli form an unbounded set — no density, frequency, or growth condition is imposed, and each instance demands a single witness. In particular, one Siegel zero is one bounded window: finding a single exceptional zero would refute GRH but would *not* satisfy the fulcrum, which demands a witness beyond every Q . The fulcrum carries no asymptotic content per window; this is the weakest shape for which the engine of §6 can run, and the demand audit certifies that *this* engine leaves nothing weaker on the table.

Remark 4.4 (The two arms of $C^\star$). Only the second arm of $C^\star = \max(C^{(1)}, 2/c_0)$ is a numeral. The first, $C^{(1)} = \exp \exp\{2A/(\mathfrak{S} C(\alpha))\}$ in the notation of [4] (its Corollary 2, with A the effective constant of its Corollary 1), is the threshold at which that engine’s error budget closes. It is a double exponential: an inner exponent as small as $2A/(\mathfrak{S} C(\alpha)) = 10$ already places $C^{(1)}$ near e^{22026} , against the reality arm’s $2/c_0 = 253696$. So $C^{(1)}$ decides the maximum, and the constant this paper exhibits as a numeral is the reality floor—the quality at or above which Theorem 4.2 derives that the witnessing zero is real—not the quality at which any engine is expected to run.

No statement here depends on the size of that gap. Theorem 5.1 quantifies C away: it is stated for every $C > 0$ and consumes the engine implication at that same C , so an engine supplying h_{Engine} at $C^{(1)}$ instantiates the dichotomy at $C^{(1)}$, with the same conclusion. The reality derivation runs in the same direction—its hypothesis is $C \geq 2/c_0$, so it applies a fortiori at any threshold above the numeral. We report 253696 because it is the arm the corpus certifies (`fulcrum_zero_real_numeral`), not as a prediction of where h_{Engine} will be supplied.

5 The dichotomy

Here *TwinPrimeConjecture* is the expected statement, in the same unbounded-threshold form as the fulcrum’s own quantifier:

$$\forall n \exists p \geq n : p \text{ and } p+2 \text{ prime}$$

(`Salt.Basic.TwinPrimeConjecture`) — the dichotomy trades one unboundedness for another.

Theorem 5.1 (The dichotomy, with the engine hypothesis explicit). *Unconditionally: if the fulcrum fails ($\neg F$), there are no Siegel zeros in the precise sense of the corpus’s `NoSiegelZeros` (`Salt.Fulcrum.not_fulcrum_implies_noSiegelZeros`). Consequently, for any $C > 0$, given the engine implication $h_{\text{Engine}} : \text{FulcrumQualityMin } C \rightarrow \text{TwinPrimeConjecture}$, we obtain*

$$\text{TwinPrimeConjecture} \vee \text{NoSiegelZeros}.$$

`Salt.Fulcrum.fulcrum_dichotomy`

Here *NoSiegelZeros* is precisely the statement

$$\begin{aligned} \exists c > 0 : \forall q > 1, \forall \chi \bmod q \text{ primitive quadratic}, \forall \beta \in \mathbb{R} : \\ L(\beta, \chi) = 0 \wedge \beta < 1 \implies \beta \leq 1 - \frac{c}{\log q}, \end{aligned}$$

i.e. some single effective constant pushes every real zero of every primitive quadratic character—all moduli, no exceptions—away from $s = 1$ at the scale $1/\log q$. Since “Siegel zero” is inherently a threshold-relative notion, this $\exists c$ form is the exact formal content of the phrase “there are no Siegel zeros” as used in the literature (it is Tao’s formulation); it is far weaker than GRH, and it speaks only of real zeros of quadratic characters—the classical exceptional configuration, which is the only one that can occur.

The second horn is thus the Heath-Brown dichotomy, machine-checked and strengthened: strengthened because the hypothesis feeding its twin horn is the minimized fulcrum rather than the 1983 hypothesis, and because the $\neg F$ horn is unconditional and effective. The engine implication h_{Engine} is the one

conditional input; §6 proves its exact-identity keystone unconditionally, leaving the remaining Heath-Brown assembly as the named campaign `HB-ENGINE`. We do not claim the unconditional dichotomy; we exhibit exactly what remains.

6 The keystone

The objects: for a primitive quadratic $\chi \bmod q$, the twisted von Mangoldt function is

$$\tilde{\Lambda}(n) := \sum_{d|n} \mu^2(d) \chi(d) \log(n/d),$$

a majorant-with-sign of Λ that agrees with it when χ is λ -like on the divisors of n ; over a finite window A ,

$$S_1(A) := \sum_{n \in A} \Lambda(n) \Lambda(n+2), \quad S_2(\chi, A) := \sum_{n \in A} \tilde{\Lambda}(n) \tilde{\Lambda}(n+2),$$

and the *pretense sum* — the price currency of the whole chapter — is

$$\text{PS}_\chi(N) := \sum_{\substack{p \leq N \\ \chi(p)=1}} \frac{\log p}{p},$$

which is $O(1)$ exactly when χ pretends to be the Liouville pattern on primes. The window $W = W(\chi, z, x)$ is the set of $n \in (x, 2x]$ with $n(n+2)$ coprime to q and to every prime $p < z$ with $\chi(p) \neq -1$: the exact sifted support on which $\tilde{\Lambda} \geq \Lambda \geq 0$ termwise, so $S_1 \leq S_2$ and the engine's content is entirely in bounding the overshoot $S_2 - S_1$.

The engine's hardest block is exactly that second-moment comparison $S_2 - S_1$ —the site of Heath-Brown's [4] Lemma 2. Formalization exposed that the classical route's τ -crude majorant is *provably* insufficient at the needed generality (its concentration pattern is unexcludable by pattern-blind tools; ledger entry #251), so the corpus proves the required conclusion directly on an exact identity: the overshoot $(\tilde{\Lambda} - \Lambda)(n) \tilde{\Lambda}(n+2) + \Lambda(n) (\tilde{\Lambda} - \Lambda)(n+2)$ is summed exactly, its support is classified, and eleven family estimates (each with explicit absolute constants; nine landed at first attempt) price every class.

Theorem 6.1 (The master estimate, unconditional). *Write $L = \log(2x+2)$. Let χ be a Dirichlet character with $\chi^2 = 1$, let $100^{16} \leq z$ and $L^8 \leq z$, let $z^3 \leq x$, and let W be the sifted window at parameters (z, x) . Then*

$$S_2(\chi, W) - S_1(W) \leq 2^{31} \left(\frac{x}{z_0} + \frac{x}{\log x} e^{5z_0} \text{PS}_\chi + e^{2z_0} (xz^{-1/8} + x^{9/10}) L^3 \right),$$

where $z_0 = \log(2x+2)/\log z$ and PS_χ is the pretense sum of χ .

`Salt.HB.hb_l2c_master_unconditional`

What briefly appeared to be a residual—a χ -weighted sieve count (the roles-swapped mirror family, where the character-positive prime is a cofactor rather than a modulus, together with one boundary family of the same shape at the engine's witness scale)—was *proven unnecessary*: both counts route through the already-established pair-count engine *character-blind*, landing in the leading budget row, so the estimate holds from its four explicit hypotheses alone. The cover of the eleven families is itself a kernel-checked theorem—no case can have been forgotten—and the three cover gaps found during execution were all found by *the machine layers below the kernel* and repaired at the statement level before any proof was attempted on a wrong statement (ledger #245, #246, #252). The regime bookkeeping produced one of the campaign's sharpest catches, an amendment refuted by its own executor at the asymptotic corner (ledger #253); at the downstream witnesses the full hypothesis packet is discharged by explicit constants (`Salt.HB.glue_master`).

7 The exchange-rate wall

The Fulcrum Hunt’s second product is negative space made precise: four recorded refutations (the *wall series*, banked in the repository’s exploration ledger) of the natural routes by which Siegel-zero information might be converted into twin-prime information. Three are ledger-grade records; the fourth is a theorem series, formalized here.

The transfer at the heart of every such route exchanges the twin sum S_1 for a twisted sum $S_2(f)$ against a multiplicative sign function f . The wall states: this exchange is *never free*, and its price is *exactly a pretense sum*—and it says so for arbitrary real sign functions, a class strictly wider than Dirichlet characters.

Definition 7.1 (Sign functions). $f : \mathbb{N} \rightarrow \mathbb{R}$ is a sign function if it is totally multiplicative with $f(1) = 1$, $|f| \leq 1$, and $f(p) = \pm 1$ on primes.

`Salt.HB.IsSignFunction`

Theorem 7.2 (Neutrality and the rate). For every sign function f : $S_1 \leq S_2(f)$ on the generalized window (unconditionally), and the difference obeys the budget

$$S_2(f) - S_1 \leq \text{EngineBound}(C_{\text{main}}, z, x, \text{PS}_f)$$

whenever the overshoot budget holds—the three-row bound of Theorem 6.1 with the character’s pretense sum replaced by f ’s.

`Salt.HB.S1_le_S2Gen`, `Salt.HB.neutrality_rate`

Theorem 7.3 (The neutrality point). The Liouville function is the exact fixed point of the exchange: $\tilde{\Lambda}_\lambda = \Lambda$ identically, $S_2(\lambda) = S_1$, the overshoot vanishes pointwise, and $\text{PS}_\lambda = 0$.

`Salt.HB.LamTildeGen_lamR_eq_vonMangoldt`, `S2Gen_lamR_eq_S1`, `overshootExactGen_lamR`,

`PretenseSumGen_lamR_eq_zero`

Read together: any route that would extract twins from a transfer must pay in pretense, the currency PS_f ; the one sign function with $\text{PS} = 0$ is the one at which the transfer returns exactly the twin sum it was fed. The wall is falsifiable—exhibit a sign function with bounded pretense sum and an S_2 -evaluation that does not route through S_1 —and it is stated over the full sign-function cone precisely so that its scope is a theorem rather than an impression. The re-typing of the entire transfer chain from Dirichlet characters to sign functions (60 declarations: the `IsSignFunction` structure, 11 definitions, and 48 proved statements) required deleting every coprimality thread; that the deletion is sound is itself part of the formal content: the character was never necessary.

8 The parity gap, as an object

The parity barrier has lived in the literature as folklore with heuristics. The corpus makes it an object with a theorem.

Definition 8.1 (Completions, parity invariance, and Z). Let $\rho(d) = \#\{0 \leq r < d : d \mid r(r+2)\}$ be the true twin local density. For a weighting $a : \mathbb{N} \rightarrow \mathbb{R}$, the type-I sum at level d and window x and its error at exponent θ are

$$A_d(a; x) := \sum_{\substack{1 \leq n \leq x \\ d \mid n(n+2)}} a(n), \quad E_\theta(a; x) := \sum_{d \leq x^\theta} \left| A_d(a; x) - \frac{\rho(d)}{d} x \right|.$$

The completion class at grade (θ, A_0) is

$$C(\theta, A_0) := \left\{ a \geq 0 : \exists C > 0, \forall x \geq 2, E_\theta(a; x) \leq \frac{Cx}{(\log x)^{A_0}} \right\}$$

— the nonnegative weightings indistinguishable from the truth by pure type-I information at level x^θ , quality $(\log x)^{-A_0}$. (The local density ρ is pinned, not quantified: completions must carry the true twin main term. Type-II bilinear information is structurally inexpressible in the class, not merely excluded.) A predicate E on weightings is parity-invariant at grade (θ, A_0) if it holds for every completion: $\text{ParityInv}(E) : \iff \forall a \in C(\theta, A_0), E(a)$ — semantic invariance over the model class, quantifying over the mechanism rather than any method. With the twin mass $T(a; x) := \sum_{n \leq x, n, n+2 \text{ prime}} a(n)$, the predicate E is twin-sufficient if for every $a \in C(\theta, A_0)$ satisfying E , the mass $T(a; \cdot)$ is unbounded. Writing **1** for the constant weighting 1 (the true sequence), the gap statement is

$$Z(\theta, A_0) : \iff \exists E \text{ twin-sufficient with } E(\mathbf{1}).$$

No Dirichlet character, L-function, or exceptional-zero oracle appears in any of these inputs; in the formal corpus this is enforced syntactically—the defining file imports nothing from the Siegel-zero or sieve tracks, checkable by its import list alone.

`Salt.Parity.Z, Salt.Parity.ParityInv (Salt/Parity/Z.lean)`

Theorem 8.2 (The gap theorem). For $0 < \theta < \frac{1}{2}$ and $A_0 \leq 2$, no predicate E is both twin-sufficient and parity-invariant. The witness is the twin-free weighting $a^-(n) = 0$ if $n, n+2$ are both prime and 1 otherwise: a Brun-grade argument (using the corpus’s explicit $\pi_2(x) \ll x/\log^2 x$ bound) places $a^- \in C(\theta, A_0)$, parity-invariance carries E from the truth to a^- , twin-sufficiency then forces $T(a^-; \cdot)$ unbounded — but $T(a^-; x) = 0$ identically.

`Salt.Parity.sufficient_true_not_parityInv`

Theorem 8.3 (Z is exactly the demand). Over the certified grade window ($0 < \theta < 1/2$, A_0 in the stated range): $Z(\theta, A_0) \iff$ the twin prime conjecture.

`Salt.Parity.Z_implies_TPC, Salt.Parity.TPC_implies_Z`

Proposition 8.4 (The census). Ten headline theorems of the corpus lie inside the parity-invariant cone: the transfer’s unconditional leg $S_1 \leq S_2$; the $k = 2$ Maynard bound $J_1 + J_2 \leq 2 \log 2 \cdot I_2$ [9] and its θ -parameterised gate failure; the impossibility that any continuous weight crosses the twin gate; the equivalence of NoSiegelZeros with the failure of infinitely many Siegel zeros; Chen’s theorem [1] (infinitely many primes p with $p+2$ a P_2 , i.e. having at most two prime factors counted with multiplicity—the standard sieve notation, in which the “at most” is exactly the parity barrier’s residue); the twin almost-prime bound $\Omega(n(n+2)) \leq 20$ infinitely often; Brun’s theorem (the twin reciprocal sum converges); the Selberg twin counting bound $\pi_2(x) = O(x/\log^2 x)$; and Chen’s second theorem (every sufficiently large even N is $p + q$ with p prime and q a P_2).

`Salt/Parity/Instances.lean` (ten declarations, one per theorem)

The three results triangulate the frontier exactly: the corpus’s censused headline theorems live inside the cone (Proposition 8.4); the twin prime conjecture is equivalent to a demand Z (Theorem 8.3); and that demand cannot be met inside the cone (Theorem 8.2). The barrier is now a boundary with coordinates. Where Z degenerates outside its certified window, the degeneracy is itself a Lean lemma beside the definition (the grade guard, `Z_trivial_of_not_completion`)—vacuity disclosed, not discovered.

9 The door-only spine

Independently of the Siegel-zero tracks, the corpus carries a full formalization of the entropy-decrement route to the logarithmic two-point Chowla statement, proved by Tao [10]; the content here is the machine-checked reduction, not the truth of the statement. Late in the campaign its residual collapsed: the prior interface (three coupled analytic witnesses) was *proven unsatisfiable* at its recorded constants—a freeze-panel finding, confirmed four independent ways, ledger #249—and the corrected budget was proven through, leaving:

Theorem 9.1 (Log-Chowla, door-only). *There exist $\delta_0 > 0$ and a regime R such that a single hypothesis—the Matomäki–Radziwiłł–Tao uniformity door MRTUniformity $R \delta$ at any $0 < \delta \leq \delta_0$ —implies the logarithmic two-point Chowla statement at R . A $\forall$ -quantified head version places the regime floor above any prescribed threshold.*

Salt.Entropy.Chowla.log_chowla_two_door_only, log_chowla_two_budget_head

Here the logarithmic two-point Chowla statement at scale x is the standard one,

$$\frac{1}{\log x} \sum_{n \leq x} \frac{\lambda(n) \lambda(n+1)}{n} \rightarrow 0,$$

rendered in the corpus as the failure of a quantified ε -deviation over the regime’s windows. The door is the Matomäki–Radziwiłł–Tao uniformity property restricted to the bounded large-spectrum frequency set Ξ_H (of size $\leq K$, independent of H): for every window length H in the regime’s range and every $\xi \in \Xi_H$,

$$\int \left\| \sum_{h \leq H} \lambda(n+h) e(-\xi h/H) \right\| d\mu_{\log}(n) \leq \delta H,$$

where $\mu_{\log}$ is the logarithmic measure of the regime — the short-interval uniformity of λ that Matomäki–Radziwiłł–Tao’s Proposition 2.4 [8] provides in prose mathematics, at finitely many frequencies only, which is all the proof fires. The door is the exact target of the corpus’s staged Matomäki–Radziwiłł program [7] (whose first stones, including the closure of a long-flagged Euler-oscillation bridge, landed the same day); the conjecture’s entire residual is thus one named seam between two formalized campaigns.

10 The Salt method

The Salt method is an adversarial division of mathematical labor: human direction and ratification; frontier-model design under statement freezes; fleet-model execution and independent refutation, tiered by difficulty class; and kernel verification as the sole referee. Its discipline—freeze before proving, refute before trusting, give up loudly, bank everything—is designed to make the system catch errors faster than it makes them.²

Language models of differing proving ability come at sharply differing inference cost, and most of a formalized proof is mechanical; the division of labor exploits exactly that asymmetry. Every lemma is classified before any proof attempt into one of four difficulty classes—*mechanical* (one tactic or a known lemma), *standard* (a routine multi-step argument), *design* (genuine proof architecture), *open*—and routed to the cheapest tier of model that can reliably discharge it: the most capable model (here Fable 5) designs the campaigns, freezes the statements, and classifies the components; *executor agents*—independent instances of a mid-tier model (here Claude Opus), dispatched one per proof obligation with a self-contained brief, hereafter *executors*—prove them at their tier. Misclassification is safe rather than fatal: an under-tiered executor is required to stop and report—give up early, loudly—and the obligation re-routes upward carrying the map of where it broke, so a grading error costs a bounded retry, never a wrong proof. Statements are frozen by design panels whose members include adversarial refuters with a standing default of *refuted*; executors are forbidden to alter a frozen statement—their duty on discovering one unprovable is to stop and report, loudly; and the house that rules on such reports is itself subject to the same audit (twice in the campaign, an executor refuted the supervising layer’s own amendment, and the record says so: ledger #248, #250).

The evidence that this discipline works is the ledger: 256 numbered catches over the campaign against *zero* wrong proofs built on any of them. The day this paper was drafted supplies the case study: five statement-layer faults surfaced in one morning—three cover-taxonomy gaps in a frozen design, one

²After water, salt.

refutation of a supervising amendment, one unsatisfiable recorded interface—and every one was caught by the layers below the kernel, ruled on at the statement level, and repaired before a single wrong proof was attempted. A companion paper (*The Salt Method*) will give the full treatment; the repository’s ledgers are the raw artifact both papers cite.

We record one measurement with a claim attached: no design error was first discovered by the kernel. Proof attempts failed and were repaired as routine, but every one of the ledger’s catches was made by the adversarial layers above the kernel, at the statement level, before a wrong proof was built on it—and no landed statement was later retracted. The kernel confirmed; the layers discovered. A verification standard is achievable at research speed precisely because correctness is enforced *above* the kernel and only certified at it.

11 The program forward

Three named campaigns continue from the corpus’s current edge, each with its target stated formally in advance: HB-ENGINE (the remaining distance from Theorem 6.1, now unconditional, to h_{Engine}); the Matomäki–Radziwiłł program toward the door of Theorem 9.1; and the explicit-constants floor (Siegel–Walfisz through Chen with every constant explicit). The conditional table below is the paper’s exact boundary.

The first of the three has since acquired coordinates, which we state because “what remains” is itself a claim this paper is obliged to make precisely. The distance from Theorem 6.1 to h_{Engine} is §§6–7 of [4]; within it the famous Lemma 10 cites exactly one result from outside the section—Estermann’s bound $S(k; u, v) \ll d(k) k^{1/2} (k, u, v)^{1/2}$ for the complete Kloosterman sum [2]—while §6, the two-variable Euler-product apparatus, is the larger share of the formalization.

Two character-sum inputs of that kind are consumed there and only one is proved in the source. The second, stated at [4, p. 217] without proof, is now a theorem of the corpus: for every primitive real character $\chi \bmod q$ and arbitrary u, u', v, v' ,

$$\left| \sum_{t \bmod q} \chi(ut + u') \chi(vt + v') \right| \leq \gcd(q, uv' - vu'),$$

with nothing assumed about the shape of q or of χ beyond primitivity; the proof runs through a structure theorem for primitive real characters that the formalization forced us to make explicit, down to the primitivity of χ_4 —which mathlib names in prose as the primitive quadratic character on $\mathbb{Z}/4$ without proving it.

`Salt.HB.sum_two_forms_le_gcd_of_isPrimitive`, `Salt.HB.structure_of_isPrimitive` Estermann’s bound the corpus supplies through its own Weil track, at present up to a factor $2^{v_2(k)/2}$: unbounded in the modulus k in general, and constant on the moduli the engine actually consumes. Closing that reduction is the named sub-campaign—and pricing the gap in a factor rather than in a citation is exactly what the formal route buys. The completion apparatus the argument needs is kernel-checked already: the sawtooth majorant’s Fourier expansion, for every real θ and every $K \geq 2$, with its coefficients bounded termwise by $K/(\pi^2 m^2)$ and in ℓ^1 by $6(1 + \log K)$.

`Salt.Weil.sawtoothMajorant_fourier_expansion`, `Salt.Weil.norm_majorantCoeff_le_sq`,
`Salt.Weil.tsum_norm_majorantCoeff_le`

Acknowledgments

This work was conducted entirely in the author’s personal capacity, on personal time and personal equipment, and used no confidential or proprietary information of any employer. The views, findings, and conclusions expressed here are the author’s own and do not represent the views, positions, or endorsement of the author’s employer or of any other organization. The collaboration with Claude (Anthropic’s

Conditional statement	Residual hypothesis	Campaign
Thm 5.1, twin horn	h_{Engine}	HB-ENGINE
Thm 9.1	the MRT door	S8 / MR program
Thm 7.2	the overshoot budget (hbudget)	§7

Table 1: Every conditional in this paper, with its exact named residual. Nothing else in the paper is conditional.

Table 5 and Opus models) is described in §10 and recorded commit-by-commit in the repository; it likewise implies no position or endorsement on the part of Anthropic. The author directed and ratified every design decision and bears sole responsibility for the claims. The formalization builds on mathlib, and this work stands on the shoulders of that community.

A The formal-statement index

Statement	Declaration	Axioms
Thm 3.1	<code>zeta_zero_free_region_littlewood</code>	the three
Thm 3.2	<code>zeta_zero_free_region_pow</code>	the three
Thm 3.3	<code>dh_repulsion_tall</code>	the three
Def 4.1	<code>FulcrumQualityMin</code>	—
Thm 4.2	<code>fulcrum_zero_real</code>	the three
Thm 5.1	<code>fulcrum_dichotomy</code>	the three
Thm 6.1	<code>hb_l2c_master_unconditional</code>	the three
Thm 7.2	<code>neutrality_rate</code>	the three
Thm 7.3	<code>S2Gen_lamR_eq_S1</code>	the three
Thm 8.2	<code>sufficient_true_not_parityInv</code>	the three
Thm 8.3	<code>Z_implies_TPC / TPC_implies_Z</code>	the three
Thm 9.1	<code>log_chowla_two_door_only</code>	the three

Table 2: “The three” = `propext`, `Classical.choice`, `Quot.sound`. Full index generated at release.

A.1 Reading convention

Each entry below pairs a numbered statement of §§3–9 with the declaration that carries it, transcribed from the source at the cited `file:line` with the proof body removed and nothing else changed: implicit binders, instance binders, coercions and numerals stand as Lean writes them, and where a section-level `variable` supplies a binder the entry says so. The definitions a statement depends on follow it, so an entry closes over its own vocabulary; anything dropped for width is named and located in the entry’s line, never silently. Every declaration here is listed in an `#audit_axioms` block, which is a build-time gate and not a `print` (`Salt/Tactic/AuditAxioms.lean:81`): it aborts elaboration, naming the offender, if a listed declaration’s axiom closure leaves `{propext, Classical.choice, Quot.sound}` or if the name does not resolve — so `lake build` succeeding is the audit passing, and each entry cites the manifest line that runs it. Names are stable across the corpus; line numbers are as of commit 244ba62 (2026-08-03), except for a minority of entries re-pinned at later commits; every pin re-extracts at one fresh commit at the release re-extraction, and the source is the arbiter wherever this rendering and the declaration could be read apart.

A.2 The four banners (§3)

Theorem 3.1 (Littlewood-strength zero-free region). Source `Salt/Vk/Littlewood.lean:409`; audit `Salt/Vk/All.lean:52`. The width $\log \log |t| / \log |t|$ is the content; the witnesses $c = 1/88214$ and $T_0 = \exp \exp(\log C + 400)$ are supplied inside the proof and are not part of the statement.

```
theorem zeta_zero_free_region_littlewood :
  ∃ c T₀ : ℝ, 0 < c ∧ 3 ≤ T₀ ∧ ∀ ρ : ℂ, riemannZeta ρ = 0 → T₀ ≤ |ρ.im| →
    ρ.re ≤ 1 - c * (Real.log (Real.log |ρ.im|) / Real.log |ρ.im|)
```

Theorem 3.2 (the power-saving region, $\theta = 3/4$). Source `Salt/Vk/GrowthPow.lean:1044`; audit `Salt/Vk/All.lean:52`. The paper’s $(\log \log |t|)^{-O(1)}$ is pinned in the declaration at the exponent 3.

```
theorem zeta_zero_free_region_pow :
  ∃ c T₀ : ℝ, 0 < c ∧ 3 ≤ T₀ ∧ ∀ ρ : ℂ, riemannZeta ρ = 0 → T₀ ≤ |ρ.im| →
    ρ.re ≤ 1 - c / ((Real.log |ρ.im|) ^ ((3 : ℝ) / 4))
    * (Real.log (Real.log |ρ.im|)) ^ (3 : ℕ)
```

Theorem 3.3 (Deuring–Heilbronn repulsion, contract form). Source `dh_repulsion_tall` at `Salt/SW/TBaITall.lean:2090`; audit `Salt/SW/All.lean:132`, the track’s roll-call invocation, which names this declaration at :346. The three constants are bound existentially in the statement; the witnesses are supplied at `Salt/SW/TBaITall.lean:2194` as $\langle 680, c, 14, \dots \rangle$, with c the explicit ten-arm minimum set at :2137. The paper’s $b = 680$ and $k = 14$ are therefore proof data, not statement data, and a reader checking the display of §3 against the declaration should read them there. The height-bounded twin of Remark 3.4 is `Salt/SW/TBaIR8.lean:1813` (witness :1921, minimum :1861): the same b and k over an eleven-arm tower whose extra arm carries the auxiliary ζ -growth constant, which is why that form does not price numerically and this one does.

```
theorem dh_repulsion_tall : ∃ b c k : ℝ, 0 < b ∧ 0 < c ∧ 0 ≤ k ∧
  ∀ (q : ℕ) [NeZero q] (χ : DirichletCharacter ℂ q),
    χ.IsPrimitive → χ ≠ 1 → χ ^ 2 = 1 → 2 ≤ q →
    ∀ β₀ : ℝ, DirichletCharacter.LFunction χ (β₀ : ℂ) = 0 → 1 / 2 < β₀ → β₀ < 1 →
    ∀ ρ : ℂ, DirichletCharacter.LFunction χ ρ = 0 → ρ.im ≠ 0 →
    16 / 17 ≤ ρ.re → ρ.re < 1 → ρ.re ≤ β₀ →
    (1 - β₀) ≥ c * ((q : ℝ) * (|ρ.im| + 2)) ^ (-(b * (1 - ρ.re)))
    / (Real.log ((q : ℝ) * (|ρ.im| + 2)) + 2) ^ k
```

Theorem 3.5 (Vinogradov mean value theorem). Source `Salt/Vmvt/Summit2.lean:151`; audit `Salt/Vmvt/All.lean:39`. The conclusion is the named predicate; its unfolding and the two constants follow, with the exponent $E(k, r)$ the quantity the source fixes and the constant $D(k, r)$ free.

```
theorem vmvt (k r x : ℕ) (hk : 2 ≤ k) (hr : 1 ≤ r) (hx : 1 ≤ x) : VmvtBound k r x

def VmvtBound (k r x : ℕ) : Prop :=
  (JkI k (k * r) x : ℝ) ≤ vmvtConst k r * (x : ℝ) ^ (vmvtExp k r)

noncomputable def JkI (k b x : ℕ) : ℕ := Jk k b (Finset.Ioc (0 : ℤ) (x : ℤ))

noncomputable def vmvtExp (k r : ℕ) : ℝ :=
  2 * (r : ℝ) * (k : ℝ) - (1 / 2) * (k : ℝ) * ((k : ℝ) + 1) + vmvtEta k r

noncomputable def vmvtEta (k r : ℕ) : ℝ := (1 / 2) * (k : ℝ) ^ 2 * (1 - 1 / (k : ℝ)) ^ r

noncomputable def vmvtConst (k r : ℕ) : ℝ := (vmvtC0 k) ^ r

noncomputable def vmvtC0 (k : ℕ) : ℝ := (k : ℝ) ^ (24 * k ^ 2)
```

A.3 The fulcrum (§4)

Definition 4.1 (the fulcrum). Source `Salt/Fulcrum/Basic.lean:61`. What is formalized is the family `FulcrumQualityMin C`; C^* is not a Lean constant, and no declaration in the corpus asserts the minimality claimed for it in §4 — that minimality is a property of the demand audit which produced the definition (the Fulcrum Hunt, recorded in the repository’s exploration ledger), and is to be read as such. The four things the definition does *not* require of ρ — reality, $\text{Re } \rho < 1$, $\text{Re } \rho > 1/2$, maximality — are visible as the absence of those conjuncts.

```

def FulcrumQualityMin (C : ℝ) : Prop :=
  ∀ Q : ℕ, ∃ (q : ℕ) (χ : DirichletCharacter ℂ q) (ρ : ℂ),
    Q < q ∧ χ.IsPrimitive ∧ χ2 = 1 ∧ χ ≠ 1 ∧
    LFunction χ ρ = 0 ∧ ||(1 : ℂ) - ρ|| * (C * Real.log q) ≤ 1

```

Theorem 4.2 (reality is derived). Source `Salt/Fulcrum/Basic.lean:93`; audit `Salt/Fulcrum/All.lean:54`. The zero-free region enters as the hypothesis `hZFR` and the quality threshold as `hC`; the conclusion is the conjunction $\text{Im } \rho = 0 \wedge 1/2 \leq \text{Re } \rho \wedge \text{Re } \rho < 1$.

```

theorem fulcrum_zero_real
  (C c₀ : ℝ) (hc₀pos : 0 < c₀) (hc₀le : c₀ ≤ 1) (hC : 2 ≤ C * c₀)
  (hZFR : ∀ (q : ℕ) [NeZero q] (χ : DirichletCharacter ℂ q), χ.IsPrimitive →
    χ ≠ 1 → ∀ {ρ : ℂ}, LFunction χ ρ = 0 → 1 / 2 ≤ ρ.re →
      (χ2 ≠ 1 ∨ ρ.im ≠ 0) →
        ρ.re ≤ 1 - c₀ / Real.log ((q : ℝ) * (|ρ.im| + 2)))
  {q : ℕ} [NeZero q] {χ : DirichletCharacter ℂ q} {ρ : ℂ}
  (hq3 : 3 ≤ q) (hprim : χ.IsPrimitive) (hne : χ ≠ 1)
  (hzero : LFunction χ ρ = 0)
  (hball : ||(1 : ℂ) - ρ|| * (C * Real.log q) ≤ 1) :
    ρ.im = 0 ∧ 1 / 2 ≤ ρ.re ∧ ρ.re < 1

```

The numeral instance, with the region supplied from the corpus at $c_0 = 1/126848$ and the threshold read off as $253696 = 2/c_0$ — the arm of C^* that Remark 4.4 reports. Sources `Salt/Fulcrum/CZeroNumeral.lean:427` and `:387`; audit `Salt/Fulcrum/All.lean:54`.

```

theorem fulcrum_zero_real_numeral (C : ℝ) (hC : 253696 ≤ C)
  {q : ℕ} [NeZero q] {χ : DirichletCharacter ℂ q} {ρ : ℂ}
  (hq3 : 3 ≤ q) (hprim : χ.IsPrimitive) (hne : χ ≠ 1)
  (hzero : LFunction χ ρ = 0)
  (hball : ||(1 : ℂ) - ρ|| * (C * Real.log q) ≤ 1) :
    ρ.im = 0 ∧ 1 / 2 ≤ ρ.re ∧ ρ.re < 1

theorem zero_free_region_all_numeral :
  ∀ (q : ℕ) [NeZero q] (χ : DirichletCharacter ℂ q), χ.IsPrimitive →
    χ ≠ 1 → ∀ {ρ : ℂ}, LFunction χ ρ = 0 → 1 / 2 ≤ ρ.re → (χ2 ≠ 1 ∨ ρ.im ≠ 0) →
      ρ.re ≤ 1 - 1 / 126848 / Real.log ((q : ℝ) * (|ρ.im| + 2))

```

A.4 The dichotomy (§5)

The three targets, verbatim: sources `Salt/Basic.lean:25`, `Salt/TwinBar/SiegelTwin.lean:76` and `:97`.

```

def TwinPrimeConjecture : Prop :=
  ∀ n : ℕ, ∃ p : ℕ, n ≤ p ∧ p.Prime ∧ (p + 2).Prime

def NoSiegelZeros : Prop :=
  ∃ c : ℝ, 0 < c ∧ ∀ (q : ℕ) [NeZero q] (χ : DirichletCharacter ℂ q),
    1 < q → χ.IsPrimitive → χ2 = 1 → χ ≠ 1 →
    ∀ β : ℝ, LFunction χ β = 0 → β < 1 →
      β ≤ 1 - c / Real.log q

def HeathBrownDichotomy : Prop :=
  TwinPrimeConjecture ∨ NoSiegelZeros

```

Theorem 5.1 (the dichotomy). Sources `Salt/Fulcrum/Dichotomy.lean:82` (the unconditional horn) and `:102` (the composite); audit `Salt/Fulcrum/All.lean:54`. The conditionality of the twin horn is a binder: `hEngine` is an argument of the theorem, so no instance of `fulcrum_dichotomy` exists without one, and Table 1’s first row is that binder.

```

theorem not_fulcrum_implies_noSiegelZeros {C : ℝ} (hC : 0 < C)
  (hnF : ¬ FulcrumQualityMin C) : NoSiegelZeros

theorem fulcrum_dichotomy {C : ℝ} (hC : 0 < C)
  (hEngine : FulcrumQualityMin C → TwinPrimeConjecture) :
    HeathBrownDichotomy

```


A.5 The keystone (§6)

Theorem 6.1 (the master estimate). Source `Salt/HB/L2cMasterUncond.lean:85`; audit `Salt/HB/All.lean:71`. The modulus is supplied by the section variable at `Salt/HB/L2cMasterUncond.lean:41`. The hypothesis packet is four items and nothing else — `hsq`, `hz100`, `hz8`, `hzx` — which is the sense in which §6 calls the estimate unconditional; the paper’s 2³¹ is `L2cCmain`.

```
theorem hb_l2c_master_unconditional (χ : DirichletCharacter ℂ q) (hsq : χ ^ 2 = 1) {z x : ℕ}
  (hz100 : 100 ^ 16 ≤ z) (hz8 : Lwin x ^ 8 ≤ z) (hzx : (z : ℝ) ^ 3 ≤ x) :
  S2 χ (l2cWindow χ z x) - S1 (l2cWindow χ z x)
    ≤ L2cCmain * ((x : ℝ) / z0 z x)
      + L2cCmain * ((x : ℝ) / Real.log x) * Real.exp (5 * z0 z x)
        * PretenseSum χ (2 * x + 2)
      + L2cCmain * Real.exp (2 * z0 z x)
        * ((x : ℝ) / (z : ℝ) ^ (1 / 8 : ℝ) + (x : ℝ) ^ ((9 : ℝ) / 10)) * Lwin x ^ 3
```

The vocabulary of that statement, in order of appearance: `Salt/HB/L2cMaster.lean:356`, `Salt/HB/L2cCore.lean:159`, `:54`, `:57`, `Salt/HB/Transfer.lean:176` and `:179`. `LamTilde` and `excPrimorial` are the character-indexed originals of the two generic definitions rendered in §A.6.

```
noncomputable def L2cCmain : ℝ := 2 ^ 31

noncomputable def l2cWindow (χ : DirichletCharacter ℂ q) (z x : ℕ) : Finset ℕ :=
  (Finset.Ioc x (2 * x)).filter (fun n => Nat.Coprime (n * (n + 2)) (q * excPrimorial χ z))

noncomputable def Lwin (x : ℕ) : ℝ := Real.log (2 * (x : ℝ) + 2)

noncomputable def z0 (z x : ℕ) : ℝ := Lwin x / Real.log z

noncomputable def S1 (A : Finset ℕ) : ℝ := ∑ n ∈ A, Λ n * Λ (n + 2)

noncomputable def S2 (χ : DirichletCharacter ℂ q) (A : Finset ℕ) : ℝ :=
  ∑ n ∈ A, LamTilde χ n * LamTilde χ (n + 2)
```

A.6 The exchange-rate wall (§7)

Definition 7.1 (sign functions). Source `Salt/HB/SignChain.lean:39`. Four fields, no modulus.

```
structure IsSignFunction (f : ℕ → ℝ) : Prop where
  map_one      : f 1 = 1
  map_mul      : ∀ a b : ℕ, f (a * b) = f a * f b
  abs_le_one   : ∀ n : ℕ, |f n| ≤ 1
  prime_pm     : ∀ p : ℕ, p.Prime → f p = 1 ∨ f p = -1
```

Theorem 7.2 (neutrality and the rate). Sources `Salt/HB/SignChain.lean:511` (the unconditional leg), `Salt/HB/SignRate.lean:42` (the budget shape) and `:53` (the rate); audit `Salt/HB/All.lean:71`. The split the paper states is the split in the signature: `S1_le_S2Gen` carries no hypothesis beyond `hf`, and the upper bound consumes `hbudget` — Table 1’s third row.

```
theorem S1_le_S2Gen (hf : IsSignFunction f) (A : Finset ℕ) : S1 A ≤ S2Gen f A

noncomputable def EngineBound (Cmain : ℝ) (z x : ℕ) (P : ℝ) : ℝ :=
  Cmain * ((x : ℝ) / z0 z x)
    + Cmain * ((x : ℝ) / Real.log x) * Real.exp (5 * z0 z x) * P
    + Cmain * Real.exp (2 * z0 z x)
      * ((x : ℝ) / (z : ℝ) ^ ((1 : ℝ) / 8) + (x : ℝ) ^ ((9 : ℝ) / 10)) * Lwin x ^ 3

theorem neutrality_rate (f : ℕ → ℝ) (hf : IsSignFunction f) (z x : ℕ) (Cmain : ℝ)
  (hbudget : ∑ n ∈ l2cWindowGen f z x, overshootExactGen f n
    ≤ EngineBound Cmain z x (PretenseSumGen f (2 * x + 2))) :
  0 ≤ S2Gen f (l2cWindowGen f z x) - S1 (l2cWindowGen f z x) ∧
  S2Gen f (l2cWindowGen f z x) - S1 (l2cWindowGen f z x)
    ≤ EngineBound Cmain z x (PretenseSumGen f (2 * x + 2))
```

Theorem 7.3 (the neutrality point). Source `Salt/HB/SignLiouville.lean:93`, `97`, `103`, `109`; audit `Salt/HB/All.lean:71`. Four statements, four lines; `lamR` is the real-valued Liouville function.

```

theorem LamTildeGen_lamR_eq_vonMangoldt (n : ℕ) : LamTildeGen lamR n = Λ n
theorem S2Gen_lamR_eq_S1 (A : Finset ℕ) : S2Gen lamR A = S1 A
lemma overshootExactGen_lamR (n : ℕ) : overshootExactGen lamR n = 0
theorem PretenseSumGen_lamR_eq_zero (N : ℕ) : PretenseSumGen lamR N = 0

```

The generic vocabulary, all from `Salt/HB/SignChain.lean` (lines 52, 80, 84, 89, 95, 100). `S1` is shared verbatim with §6: the transfer is re-typed on one side of the comparison only.

```

noncomputable def LamTildeGen (f : ℕ → ℝ) (n : ℕ) : ℝ :=
  ∑ d ∈ n.divisors, (μ d : ℝ) ^ 2 * f d * Real.log ((n / d : ℕ) : ℝ)

noncomputable def S2Gen (f : ℕ → ℝ) (A : Finset ℕ) : ℝ :=
  ∑ n ∈ A, LamTildeGen f n * LamTildeGen f (n + 2)

noncomputable def overshootExactGen (f : ℕ → ℝ) (n : ℕ) : ℝ :=
  (LamTildeGen f n - Λ n) * LamTildeGen f (n + 2) + Λ n * (LamTildeGen f (n + 2) - Λ (n + 2))

noncomputable def PretenseSumGen (f : ℕ → ℝ) (N : ℕ) : ℝ :=
  ∑ p ∈ (Finset.range (N + 1)).filter (fun p => Nat.Prime p ∧ f p = 1), Real.log (p : ℝ) / p

noncomputable def excPrimorialGen (f : ℕ → ℝ) (z : ℕ) : ℕ :=
  ∏ p ∈ (Finset.range z).filter (fun p => p.Prime ∧ f p ≠ -1), p

noncomputable def l2cWindowGen (f : ℕ → ℝ) (z x : ℕ) : Finset ℕ :=
  (Finset.Ioc x (2 * x)).filter (fun n => Nat.Coprime (n * (n + 2)) (excPrimorialGen f z))

```

A.7 The parity gap (§8)

Definition 8.1 (completions, parity invariance, Z). Source `Salt/Parity/Z.lean:44--103`, in file order. Two features claimed in §8 are checkable here by inspection: `typeISum` is the only functional through which `Completion` reads the weighting, so no bilinear datum is expressible in the class; and `twinRho` is a fixed function, not a quantified one, so every completion carries the same main term.

```

def twinRho (d : ℕ) : ℕ :=
  ((Finset.range d).filter (fun r => d | r * (r + 2))).card

noncomputable def typeISum (a : ℕ → ℝ) (d x : ℕ) : ℝ :=
  ∑ n ∈ Finset.Icc 1 x, if d | n * (n + 2) then a n else 0

noncomputable def typeIError (a : ℕ → ℝ) (θ : ℝ) (x : ℕ) : ℝ :=
  ∑ d ∈ Finset.Icc 1 ⌊(x : ℝ) ^ θ⌋+,
    |typeISum a d x - (twinRho d : ℝ) / d * x|

def Completion (θ A₀ : ℝ) (a : ℕ → ℝ) : Prop :=
  (∀ n, 0 ≤ a n) ∧
  ∃ C : ℝ, 0 < C ∧ ∀ x : ℕ, 2 ≤ x →
    typeIError a θ x ≤ C * x / Real.log x ^ A₀

def ParityInv (θ A₀ : ℝ) (E : (ℕ → ℝ) → Prop) : Prop :=
  ∀ a : ℕ → ℝ, Completion θ A₀ a → E a

noncomputable def twinMass (a : ℕ → ℝ) (x : ℕ) : ℝ :=
  ∑ n ∈ Finset.Icc 1 x, if n.Prime ∧ (n + 2).Prime then a n else 0

def TwinSufficient (θ A₀ : ℝ) (E : (ℕ → ℝ) → Prop) : Prop :=
  ∀ a : ℕ → ℝ, Completion θ A₀ a → E a →
  ∀ C : ℝ, ∃ x : ℕ, C < twinMass a x

def oneWeight : ℕ → ℝ := fun _ => 1

noncomputable def twinFree : ℕ → ℝ :=
  fun n => if n.Prime ∧ (n + 2).Prime then 0 else 1

def Z (θ A₀ : ℝ) : Prop :=
  ∃ E : (ℕ → ℝ) → Prop, TwinSufficient θ A₀ E ∧ E oneWeight

```

Theorem 8.2 (the gap theorem). Source `Salt/Parity/Z.lean:678`; audit `Salt/Parity/All.lean:22`. The grade condition is the single binder `h2: A₀ ≤ 2`. Until 2026-08-11 this theorem also carried `h1`

$(1 \leq A_0)$ and $\text{ht } (E(1))$; writing its comprehensibility certificate showed both were unused by the proof, and they were dropped — the kernel had been holding a strictly stronger statement than the one displayed.

```
theorem sufficient_true_not_parityInv {θ A₀ : ℝ} (h₀ : 0 < θ) (h : θ < 1 / 2)
  (h₂ : A₀ ≤ 2) {E}
  (hs : TwinSufficient θ A₀ E) : ¬ ParityInv θ A₀ E
```

Theorem 8.3 (Z is exactly the demand). Sources `Salt/Parity/Z.lean:687` and `:216`; audit `Salt/Parity/All.lean:22`. The forward direction is certified on $0 < \theta < 1/2$, $0 \leq A_0$; the converse carries no grade hypothesis.

```
theorem Z_implies_TPC {θ A₀ : ℝ} (h₀ : 0 < θ) (h : θ < 1 / 2) (hA : 0 ≤ A₀) :
  Z θ A₀ → TwinPrimeConjecture
```

```
theorem TPC_implies_Z {θ A₀ : ℝ} : TwinPrimeConjecture → Z θ A₀
```

The grade guard of §8, which states the degeneracy rather than leaving it to be found. Source `Salt/Parity/Z.lean:125`; audit `Salt/Parity/All.lean:22`.

```
theorem Z_trivial_of_not_completion {θ A₀ : ℝ}
  (h : ¬ Completion θ A₀ oneWeight) : Z θ A₀
```

Proposition 8.4 (the census). Source `Salt/Parity/Instances.lean`, ten declarations at lines 37, 45, 53, 62, 72, 79, 86, 93, 99, 106: `parityInv_S1_le_S2`, `parityInv_twin_bar`, `parityInv_twin_gate_fails`, `parityInv_no_twin_weight`, `parityInv_noSiegel_iff`, `parityInv_chen_headline`, `parityInv_twin_almost_prime`, `parityInv_N6_2`, `parityInv_N5_3`, `parityInv_chen_second`. Six of the ten are named in the audit block at `Salt/Parity/All.lean:22`; the four that are not are `parityInv_twin_gate_fails`, `parityInv_no_twin_weight`, `parityInv_noSiegel_iff` and `parityInv_N6_2`. Each of the ten has the shape below — the closed corpus theorem wrapped by `parityInv_of_closed` (`Salt/Parity/Z.lean:117`) — so the census is one lemma about where a closed proposition sits, instantiated ten times.

```
theorem parityInv_S1_le_S2 (θ A₀ : ℝ) :
  Salt.Parity.ParityInv θ A₀
  (fun _ => ∀ (q : ℕ) (χ : DirichletCharacter ℂ q), χ ^ 2 = 1 →
    ∀ A : Finset ℕ, Salt.HB.S1 A ≤ Salt.HB.S2 χ A)
```

A.8 The door-only spine (§9)

Theorem 9.1 (log-Chowla, door-only). Sources `Salt/Entropy/Chowla/SpineFinal.lean:981` (the terminal) and `:750` (the $\forall$ -quantified head); audit `Salt/Entropy/Chowla/All.lean:384`. The single residual is the door binder — Table 1’s second row — and the head’s `extraFloor` quantifier is what places the regime floor above a prescribed threshold.

```
theorem log_chowla_two_door_only :
  ∃ (δ₀ : ℝ) (R : ChowlaRegime), 0 < δ₀ ∧
    ∀ δ : ℝ, 0 < δ → δ ≤ δ₀ → MRTUniformity R δ →
      ¬ logChowla2Fails R.εps R.x R.ω

theorem log_chowla_two_budget_head :
  ∃ (ε : ℚ) (δ₀ : ℝ), 0 < ε ∧ 0 < δ₀ ∧
    ∀ extraFloor : ℕ, ∃ R : ChowlaRegime, R.εps = ε ∧ extraFloor ≤ R.Hlo ∧
      ∀ δ : ℝ, 0 < δ → δ ≤ δ₀ → MRTUniformityXi R δ →
        ¬ logChowla2Fails R.εps R.x R.ω
```

The door, its Ξ_H -restriction (the form the terminal consumes before weakening), and the failure predicate the theorems negate. Sources `Salt/Entropy/Chowla/MRTDoor.lean:48`, `:109` and `Salt/Entropy/Chowla/ChowlaFailure.lean:59`. The negated predicate is Tao’s two-point statement at shift 1: the display of §9 is `logChowla2Fails` with the inequality reversed.

```
noncomputable def MRTUniformity (R : ChowlaRegime) (δ : ℝ) : Prop :=
  ∀ H : ℕ, R.Hlo ≤ H → H ≤ R.Hhi → ∀ α : ℝ,
    (∫ n, ||windowExpSum H n α|| ∂(logMeasure R.x R.ω)) ≤ δ * (H : ℝ)

noncomputable def MRTUniformityXi (R : ChowlaRegime) (δ : ℝ) : Prop :=
  ∀ H : ℕ, ∀ [NeZero H], R.Hlo ≤ H → H ≤ R.Hhi → ∀ ξ ∈ bigXi R.εps H,
```

```

( $\int$  n,  $\|\text{windowExpSum } H \ n \ (-(\xi.\text{val} : \mathbb{R}) / (H : \mathbb{R}))\| \partial(\log\text{Measure } R.x \ R.\omega) \leq \delta * (H : \mathbb{R})$ )
def logChowla2Fails (eps :  $\mathbb{Q}$ ) (x  $\omega$  :  $\mathbb{N}$ ) : Prop :=
  (eps :  $\mathbb{R}$ ) * Real.log ( $\omega$  :  $\mathbb{R}$ ) <
     $\sum n \in \text{Finset.Ioc } (x / \omega) \ x,$ 
    (ArithmeticFunction.liouville n :  $\mathbb{R}$ )
    * (ArithmeticFunction.liouville (n + 1) :  $\mathbb{R}$ ) / (n :  $\mathbb{R}$ )|

```

The regime carrying the scales, data fields only. Source `Salt/Entropy/Chowla/Regime.lean:56`; the structure’s remaining 19 fields (lines 73–141, `hx` through `hxbig`) are proof obligations discharged at construction, elided here for width.

```

structure ChowlaRegime where
  x :  $\mathbb{N}$ 
   $\omega$  :  $\mathbb{N}$ 
  a :  $\mathbb{N}$ 
  eps :  $\mathbb{Q}$ 
  Hlo :  $\mathbb{N}$ 
  Hhi :  $\mathbb{N}$ 
  CO :  $\mathbb{N}$ 
  J :  $\mathbb{N}$ 

```

A.9 Inherited infrastructure (§3)

The two unconditional almost-prime statements §3 cites. Sources `Salt/Fulcrum/ChenCorollary.lean:34` (audit `Salt/Fulcrum/All.lean:54`) and `Salt/BrunLower/TwinInstance.lean:771` (audit `Salt/BrunLower/All.lean:54`); `cardFactors` is `mathlib`’s Ω .

```

theorem chen_omega_prod_le_three :
  {p :  $\mathbb{N}$  | p.Prime  $\wedge$  cardFactors (p * (p + 2))  $\leq$  3}.Infinite

theorem twin_almost_prime :
  {n :  $\mathbb{N}$  | ArithmeticFunction.cardFactors (n * (n + 2))  $\leq$  20}.Infinite

```

Funding statement This work received no external funding.

Competing interests The author declares none. The capacity and employer-independence disclosures in the Acknowledgments apply.

Data availability statement The complete Lean 4 corpus underlying every theorem, the commit ledger, and the priority surveys cited in the text are publicly available at <https://github.com/jyh/salt>; Appendix A maps each stated result to its Lean declaration and axiom audit.

Author contributions Developed in collaboration with Claude (Anthropic’s Fable 5 and Opus models); see §10 and the repository’s commit ledger, in which every commit is co-authored. Written in a personal capacity.

References

- [1] J. R. Chen, *On the representation of a larger even integer as the sum of a prime and the product of at most two primes*, Sci. Sinica **16** (1973), 157–176.
- [2] T. Estermann, *On Kloosterman’s sum*, Mathematika **8** (1961), 83–86.
- [3] T. Hales et al., *A formal proof of the Kepler conjecture*, Forum of Mathematics, Pi **5** (2017), e2.
- [4] D. R. Heath-Brown, *Prime twins and Siegel zeros*, Proc. London Math. Soc. (3) **47** (1983), 193–224.

- [5] J. E. Littlewood, *Researches in the theory of the Riemann ζ -function*, Proc. London Math. Soc. (2) **20** (1922).
- [6] The mathlib community, *The Lean mathematical library*, CPP 2020.
- [7] K. Matomäki and M. Radziwiłł, *Multiplicative functions in short intervals*, Ann. of Math. **183** (2016).
- [8] K. Matomäki, M. Radziwiłł, and T. Tao, *An averaged form of Chowla's conjecture*, Algebra Number Theory **9** (2015).
- [9] J. Maynard, *Small gaps between primes*, Ann. of Math. **181** (2015), 383–413.
- [10] T. Tao, *The logarithmically averaged Chowla and Elliott conjectures for two-point correlations*, Forum Math. Pi **4** (2016).